

```

    } heapify(a, i);
    for (i=0; i<n; i++) {
        printf("%d", a[i]);
    }
    return 0;
}

```

07/07/21

## Graph:-

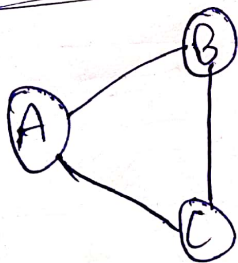
Graph is a non-linear data structure  
it is a set of vertex & edge  $G(V, E)$ .

\* Vertex is node.

\* Edge is connection between two vertex.

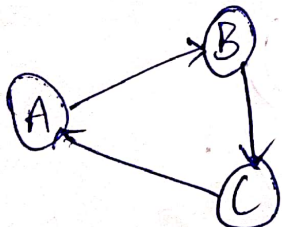
## Types of Graph

### ① Undirected graph

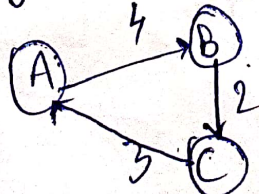


⇒ There can be any directional flow.

### ② Directed Graph



### ③ Weighted Graph

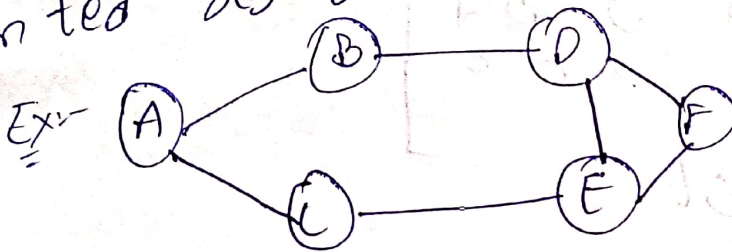


## Graph Representation

- ① Adjacent Matrix (Sparse matrix)
- ② Adjacent List
- ③ Transitive closure.

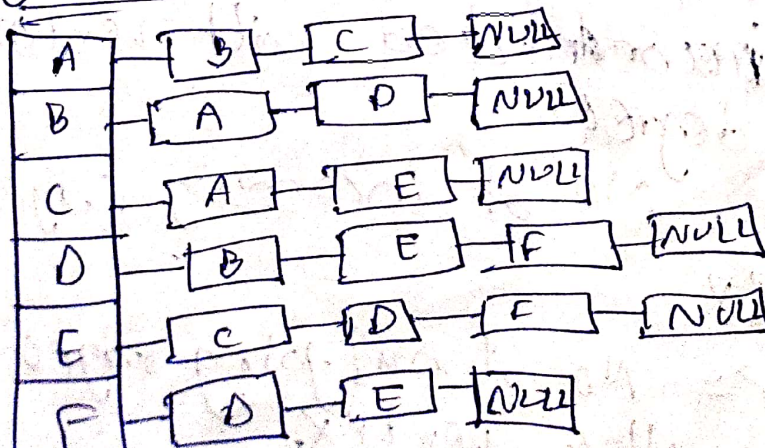
### Adjacent Matrix

→ A vertex connected with another one is represented as '1' otherwise it is represented as '0'.

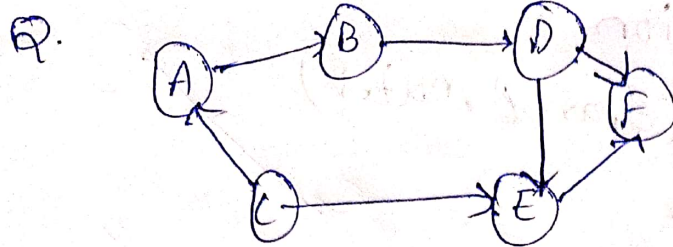


|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 1 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 0 | 1 | 0 | 0 | 1 | 1 |
| E | 0 | 0 | 1 | 1 | 1 | 0 |
| F | 0 | 0 | 0 | 1 | 1 | 0 |

### Adjacent List



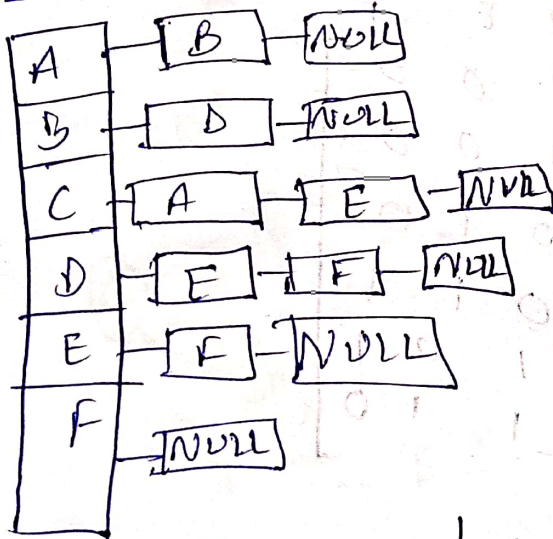




### Adjacent Matrix

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 0 | 1 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 1 | 0 |
| D | 0 | 0 | 0 | 0 | 1 | 1 |
| E | 0 | 0 | 0 | 0 | 0 | 1 |
| F | 0 | 0 | 0 | 0 | 0 | 0 |

### Adjacent List



### Degree of the vertex

No of vertices connected with vertex is called degree.

→ In degree = No of incoming edges to the vertex.

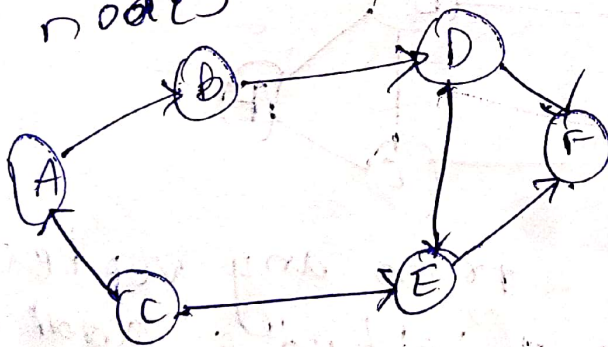
→ Out degree = No of outgoing edges from the vertex.

|   | In | Out |
|---|----|-----|
| A | 1  | 1   |
| B | 1  | 1   |
| C | 0  | 2   |
| D | 1  | 2   |
| E | 2  | 1   |
| F | 2  | 0   |

## Transitive Closure

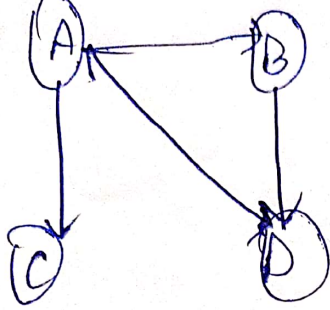
\* The path between one vertex to another vertex or reachability from one vertex to another vertex is called transitive closure.

Note:- There is a path of self-visiting nodes.



|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 1 | 1 | 0 | 1 | 1 | 1 |
| B | 0 | 1 | 0 | 1 | 1 | 1 |
| C | 1 | 1 | 1 | 1 | 1 | 1 |
| D | 0 | 0 | 0 | 1 | 1 | 1 |
| E | 0 | 0 | 0 | 0 | 1 | 1 |
| F | 0 | 0 | 0 | 0 | 0 | 1 |





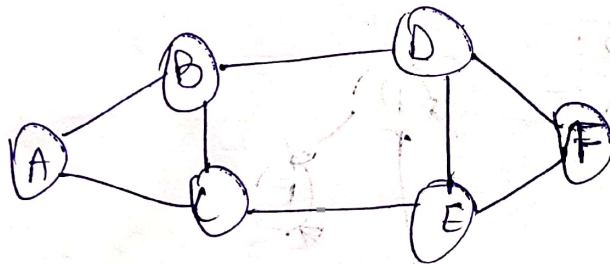
|   | A | B | C | D | <del>E</del> |
|---|---|---|---|---|--------------|
| A | 1 | 1 | 1 | 1 |              |
| B | 1 | 1 | 1 | 1 |              |
| C | 1 | 1 | 1 | 1 |              |
| D | 1 | 1 | 1 | 1 |              |

## Graph Traversal

- ① BFS (Breadth First Search)  $\Rightarrow$  Queue
- ② DFS (Depth first Search)  $\Rightarrow$  Stack

### BFS

Ex:-



Step 1:- Start from any vertex & consider it <sup>as</sup> visited & add into ~~queue~~ <sup>unvisited</sup> queue.

Step 2:- Explore all the adjacent vertex of the node and add into queue.

Visiting Order  $\Rightarrow$  A, B, C, D, E, F (BFS)

[A] [B] [C] [D] [E] [F]

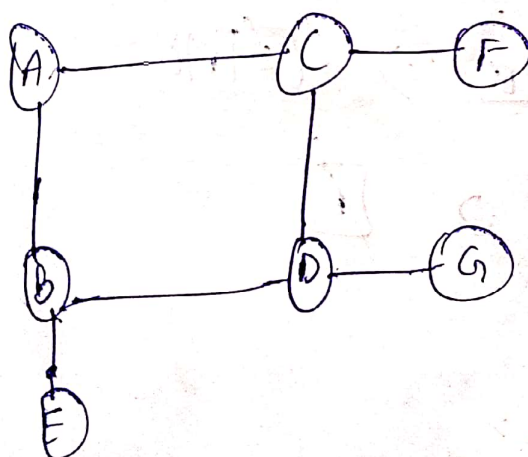
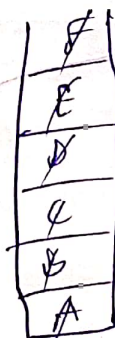
DFS

visiting order =

① Start from any vertex push into stack and consider as visited.

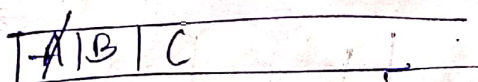
② Explore any one unvisited adjacent vertex and push into stack and consider it as visited.

DFS  $\Rightarrow$  A, B, C, E, D, F



visiting order = A, B, C, E, D, F, G

BFS  $\Rightarrow$  A, B, C



$\Rightarrow$

BFS  $\Rightarrow$  A, B, C, E, D

$\Rightarrow$

BFS  $\Rightarrow$  A, B, C, E, D, F

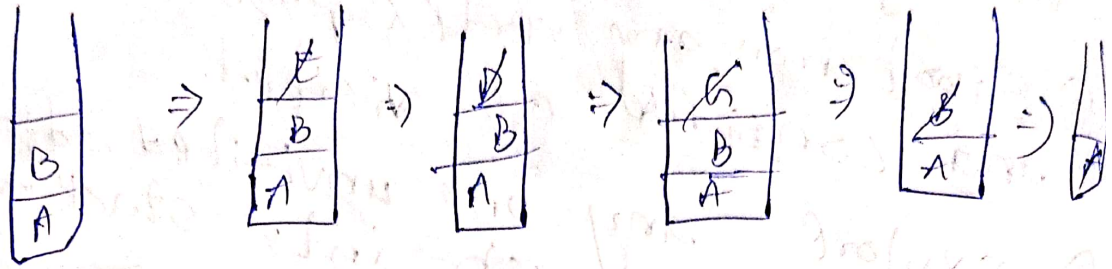
BFS  $\rightarrow$  A, B, C, E, D, F

$\Rightarrow$

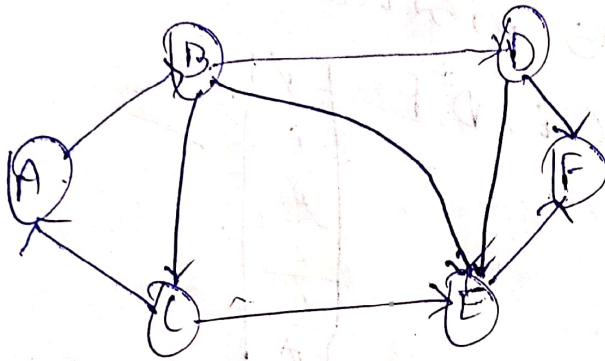
BFS  $\rightarrow$  A, B, C, E, D, F, G



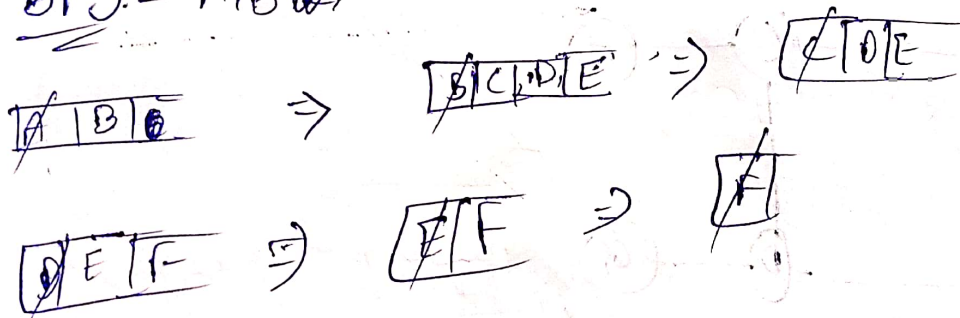
DFS:- A, B, E, D, G, C, F



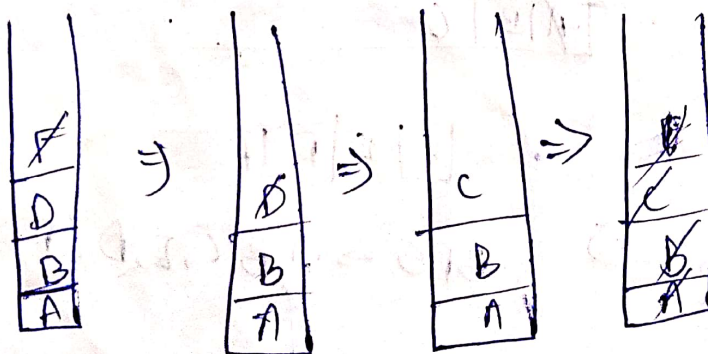
EX:-



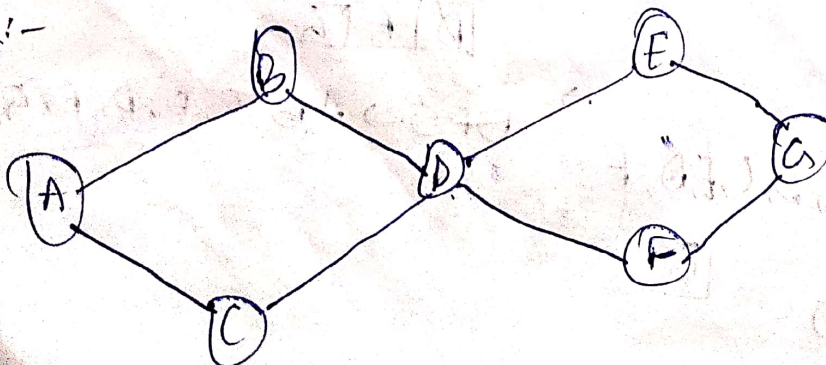
BFS:- A, B, C, D, E, F

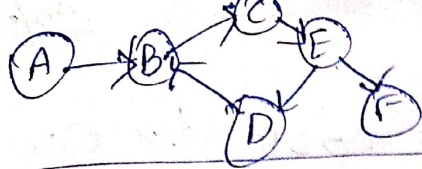


DFS:- A, B, D, F, E, C



EX:-





④ B.F.S = A, B, C, D, E, F, G

~~A~~ | B | C ⇒ ~~B~~ | C | D ⇒ ~~C~~ | D

~~D~~ | E | F ⇒ ~~E~~ | F | G ⇒ ~~F~~ | G

D.F.S with 'D' as starting point

B.F.S = D, B, C, E, F, A, G

~~D~~ | B | C | E | F ⇒ ~~B~~ | C | E | F | A

⇒ ~~C~~ | E | F | A ⇒ ~~E~~ | F | A | G ⇒ ~~F~~ | A | G

~~A~~ | G ⇒ ~~G~~

D.F.S = D, B, A, C, E, G, F

~~A~~  
A  
B  
D

⇒ ~~A~~  
B  
D

⇒ ~~B~~  
D

⇒ ~~F~~  
G  
E  
D

⇒ ~~G~~  
E  
D

⇒ ~~D~~

⇒ ~~D~~

⑤ B.F.S = A, B, C, E, F, D

~~A~~ | B ⇒ ~~B~~ | C ⇒ ~~C~~ | E ⇒ ~~E~~ | F

~~F~~ ⇒ ~~E~~ | D ⇒ ~~D~~

D.F.S = A, B, C, E, D, F

~~E~~  
C  
D  
A

⇒ ~~E~~  
C  
B  
A

⇒ ~~F~~  
E  
C  
B  
A

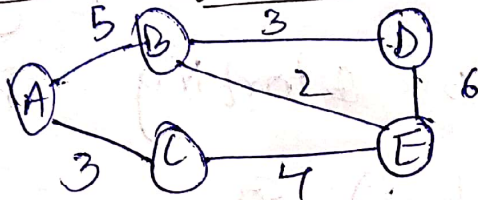




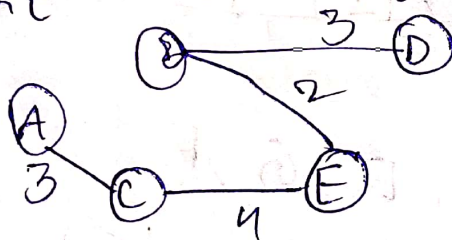
## Minimum Spanning Tree

The minimum possible cost to connect all the vertices without forming cycle is called M.S.T.

### Prim's Algorithm



- 1) Start with any vertex
- 2) Explore the selected vertex & select minimum weighted unlisted adjacent vertex without forming cycle.

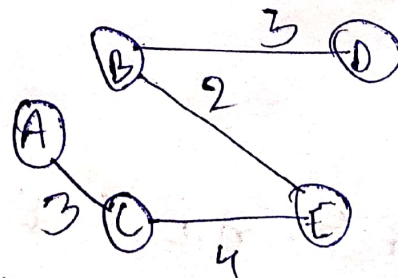


MST: 12

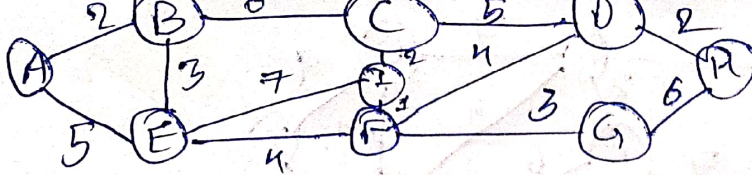
### Kruskal's Algorithm

To sort all the edges in descending order and select minimum weight without forming cycle.

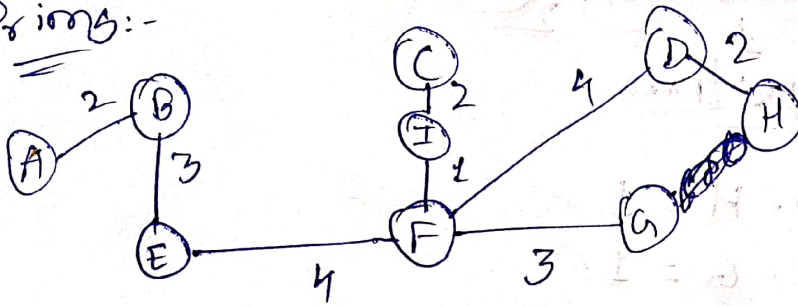
|         |           |
|---------|-----------|
| A-B = 5 | B-E = 2 ✓ |
| A-C = 3 | A-C = 3 ✓ |
| C-E = 4 | B-D = 3 ✓ |
| B-E = 2 | C-E = 4 ✓ |
| B-D = 3 | A-B = 5   |
| D-E = 6 | D-E = 6   |



MST: 12



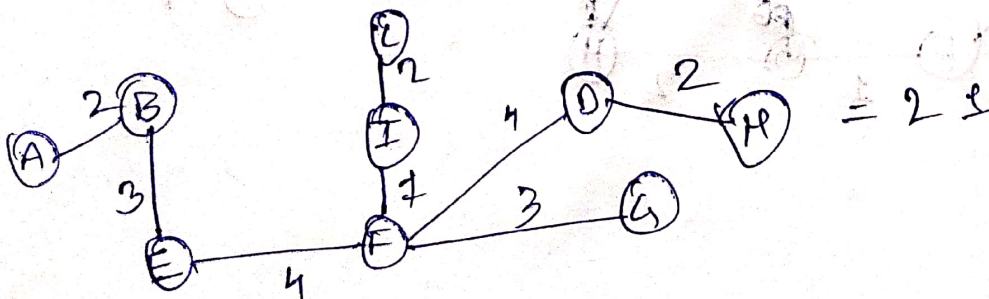
Prims:-



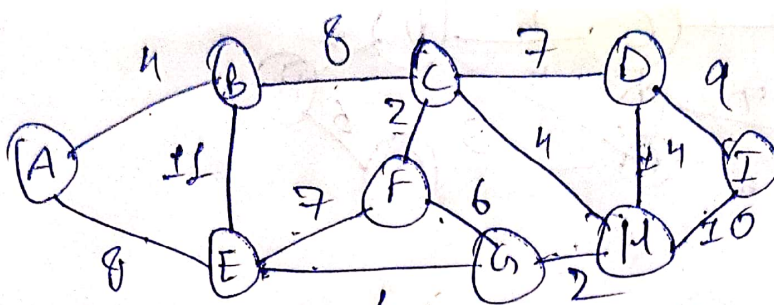
MST : 21

Kruskal's Algorithm

- |           |           |
|-----------|-----------|
| A-B = 2 ✓ | F-I = 1 ✓ |
| A-E = 5 ✓ | A-B = 2 ✓ |
| B-E = 3 ✓ | C-I = 2 ✓ |
| B-C = 6 ✓ | D-H = 2 ✓ |
| C-I = 2 ✓ | B-E = 3 ✓ |
| C-D = 5 ✓ | F-G = 3 ✓ |
| D-H = 2 ✓ | E-F = 4 ✓ |
| D-F = 4 ✓ | D-F = 4   |
| E-I = 7 ✓ | F-D = 4 ✓ |
| E-F = 4 ✓ | A-E = 5   |
| F-I = 1 ✓ | C-D = 5   |
| F-D = 4 ✓ | B-C = 6   |
| F-G = 3 ✓ | G-H = 6   |
| G-H = 6 ✓ | E-I = 7   |

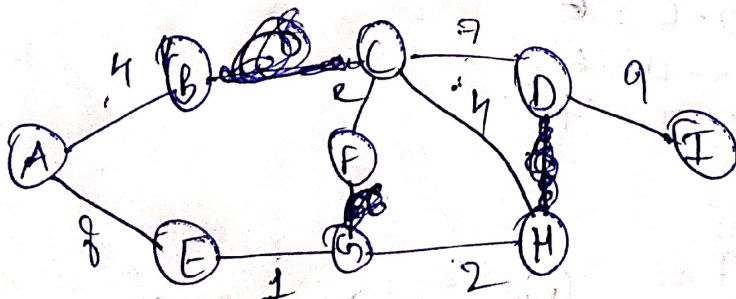






## Kruskal's Algorithm

|                      |            |
|----------------------|------------|
| A-B = 4 ✓            | D-H = 1 ✓  |
| A-E = 8 ✓            | E-G = 1 ✓  |
| B-E = 11 ✓           | G-H = 2 ✓  |
| B-C = 8 ✓            | C-F = 2 ✓  |
| C-D = 7 ✓            | A-B = 4 ✓  |
| C-F = 2 ✓            | C-H = 4 ✓  |
| <del>C-H = 4 ✓</del> | F-G = 6 ✓  |
| D-H = 1 ✓            | C-D = 7 ✓  |
| D-I = 9 ✓            | E-F = 7 ✓  |
| E-F = 7 ✓            | A-E = 8 ✓  |
| E-G = 1 ✓            | B-C = 8 ✓  |
| F-G = 6 ✓            | D-I = 9 ✓  |
| G-H = 2 ✓            | H-I = 10 ✓ |
| H-I = 10 ✓           | B-E = 11 ✓ |



$$\begin{aligned}
 &4 + 1 + 2 + 2 + 4 + \\
 &7 + 9 + \dots \\
 &= 37
 \end{aligned}$$